



AKADEMIN FÖR TEKNIK OCH MILJÖ
Avdelningen för datavetenskap och samhällsbyggnad

Laboration 5

Robin Nymoen
2026



Inledning

Syftet med denna laboration var att implementera en enkel grafstruktur i Java bestående av vertexer och kanter. Grafstrukturen kan användas för att representera nätverk, vägkartor och andra typer av sammankopplade objekt. Ett andra syfte med laborationen var att studera Dijkstras algoritm för att undersöka hur grafstrukturen skulle kunna användas för att hitta kortaste vägar mellan noder i en graf.

Uppgift 1 – Implementation av graf

I denna uppgift implementerades tre centrala klasser: `Vertex`, `Edge` och `Graph`. Dessutom användes det färdiga interfacet `GraphInterface` som grund för grafens funktionalitet.

Klassen `Vertex` representerar en nod i grafen. Varje vertex innehåller ett informationsvärde av typen `T`, två koordinater samt en färg. Informationsvärdet används som identifierare för noden medan koordinaterna beskriver dess position i ett tvådimensionellt plan.

Klassen `Edge` representerar en riktad kant mellan två vertexar. Varje kant innehåller en startvertex, en slutvertex, en distans och en färg. Distansen beräknas automatiskt när kanten skapas genom att använda den euklidiska distansen mellan vertexarnas koordinater.

`Graph` är huvudklassen i implementationen och ansvarar för att lagra grafens vertexar och kanter. Vertexarna lagras i en `HashMap` där informationsvärdet används som nyckel. Kanterna lagras i en separat `HashMap` där varje vertex är kopplad till en lista med sina utgående kanter. Detta motsvarar en implementation med adjacency list. När en ny vertex läggs till kontrolleras först om identifieraren redan finns i grafen. Om en vertex med samma identifierare redan existerar sker ingen insättning. Detta gör att varje vertex endast kan förekomma en gång i grafen. Grafen hanterar oriktade kanter genom att lagra två riktade `Edge`-objekt, ett i vardera riktningen. På så sätt kan grafen traverseras från båda hållen samtidigt som implementationen förblir enkel.

Tester

Samtliga publika metoder i `Graph`-klassen testades med hjälp av Junit-tester. Testerna kontrollerar att vertexar och kanter läggs till korrekt, att dubletter ignoreras samt att borttagning av vertexar fungerar som förväntat. Flera tester skrevs för att kontrollera att grafen hanterar felaktiga operationer korrekt. Exempel på detta är försök att skapa kanter mellan vertexar som inte existerar, att skapa kanter från en vertex till sig själv samt att ta bort vertexar som saknas i grafen. Testerna verifierar även att en tom graf innehåller noll vertexar och noll kanter samt att en isolerad vertex kan tas bort utan att påverka grafens struktur.

Ideérna för testerna och namnen på testerna gjordes med hjälp av ChatGPT, till stor del för att hitta olika edge-cases som annars skulle kunna missas. Själva koden är dock självskriven baserad på testerna som ChatGPT kom fram till. Det som blev uppenbart är att testernas namn blir ganska långa och otympliga och en möjlig förbättring är att korta ner metodnamnen för testerna. Namnen behövs dock som dom är, mest för att demonstrera potentiella problem med att använda LLM's i programmeringssyfte.

Uppgift 2 – Dijkstras algoritm

Dijkstras algoritm används för att hitta de kortaste vägarna från en startnod till alla andra noder i en viktad graf där kantvikterna är icke-negativa (W3Schools, 2026). Algoritmen används bland annat inom navigationssystem, nätverksrouting och andra tillämpningar där den kortaste vägen mellan noder behöver beräknas.

Algoritmen börjar med att ge startnoden avståndet 0 medan alla övriga noder tilldelas ett oändligt stort avstånd. Därefter väljs den obesökta nod som för tillfället har det minsta kända avståndet från startnoden. För varje granne till den valda noden beräknas ett nytt möjligt avstånd genom att lägga samman det aktuella avståndet med kantens vikt. Om detta avstånd är kortare än det tidigare registrerade avståndet uppdateras nodens information. Processen upprepas tills alla noder har behandlats eller tills inga fler närliggande noder återstår (W3Schools, 2026).

En viktig egenskap hos algoritmen är att när en nod har valts som den med lägst känt avstånd kan detta avstånd betraktas som slutgiltigt. Detta fungerar eftersom algoritmen förutsätter att alla kantvikter är icke-negativa (W3Schools, 2026).

Tidskomplexiteten beror på vilka datastrukturer som används. En enkel implementation utan prioritetskö har tidskomplexiteten $O(V^2)$, där V är antalet vertexar i grafen. Om algoritmen istället implementeras med en prioritetskö som en min-heap kan tidskomplexiteten förbättras till $O((V + E) \log V)$, där E är antalet kanter (W3Schools, 2026).

Den grafstruktur som implementerades i denna laboration använder adjacency lists, vilket gör den väl lämpad för en framtida implementation av Dijkstras algoritm. När en nod behandlas kan dess grannar hämtas direkt från grafens kantlistor utan att hela grafen behöver traverseras.

Skulle du lägga till några attribut till Graph, Edge och Vertex?

För att kunna genomföra Dijkstras algoritm behövs ingen ändring av attribut i någon av klasserna. Grafen innehåller redan all information som krävs för att representera noder och kanter samt deras kopplingar.

I Edge-klassen finns redan attributet distance som representerar kantens vikt, vilket används direkt av algoritmen. Graph-klassen hanterar redan allt som behövs för traversal av grafen via vertexar och adjacency lists. Även Vertex-klassen behöver inga extra attribut eftersom information om kortaste avstånd, föregående nod och besökta noder istället borde hanteras av en separat implementation av Dijkstras-algoritm.

Skulle du lägga till några metoder?

Jag skulle inte lägga till några Dijkstra-relaterade metoder direkt i Graph-klassen eftersom Graph ska fungera som en generell datastruktur för att lagra noder och kanter. Istället bör Graph endast erbjuda grundläggande metoder för att hantera och hämta data, som att lägga till och hämta vertexar och kanter. Själva Dijkstras algoritm bör implementeras i en separat klass som använder Graph via dess publika metoder. På så sätt hålls grafstrukturen oberoende av specifika algoritmer och kan återanvändas för andra algoritmer.

Vad skulle typen T vanligen kunna vara för en sådan implementation?

Typen T representerar informationen som lagras i varje vertex. I geografiska tillämpningar skulle detta exempelvis kunna vara ortsnamn, vägkorsningar, adress-ID eller andra unika identifierare. I ett GIS-system skulle T kunna innehålla namn på städer, koordinatpunkter eller objekt i ett vägnät. Eftersom grafen använder T som nyckel i sina HashMap-strukturer är det viktigt att varje värde är unikt. Algoritmen skulle fortfarande fungera utan informationsvärdet, men grafen blir betydligt mer användbar om varje nod kan kopplas till verklig information.

Hur skulle man kunna använda klassernas metoder för att visualisera algoritmens utmatning?

Eftersom både Vertex och Edge innehåller färgattribut kan dessa användas för att visualisera algoritmens resultat. Besökta noder skulle exempelvis kunna markeras med blå färg medan den slutliga kortaste vägen markeras med gröna kanter och gröna noder. På detta sätt skulle användaren kunna följa hur algoritmen arbetar genom grafen.

Metoderna getAllVertices() och getEdges() gör det möjligt för ett grafiskt gränssnitt att hämta alla noder och kanter och sedan rita ut dem på skärmen. När algoritmen avslutas kan färgerna uppdateras för att visa resultatet visuellt.

Slutsats

Laborationen resulterade i en fungerande implementation av en generisk grafstruktur i Java. Grafen består av vertexar och kanter samt stödjer insättning, borttagning och hämtning av grafens innehåll. Genom att använda HashMap och adjacency lists kan grafen lagra kopplingar mellan noder på ett effektivt sätt. Implementationen testades med JUnit för att säkerställa att samtliga publika metoder fungerar korrekt.

Den teoretiska delen av laborationen visade även hur grafstrukturen skulle kunna användas tillsammans med Dijkstras algoritm för att hitta kortaste vägarna i ett nätverk. Grafen skulle med relativt små förändringar kunna byggas ut för att stödja en fullständig implementation av algoritmen.

Referenser

W3Schools. (2026). DSA Dijkstra's Algorithm. Hämtad 2026-06-09 från https://www.w3schools.com/dsa/dsa_algo_graphs_dijkstra.php